CITS5501 Project 2026 — Phase 2 Report

Group Number — 14 | Group Name — Scrapers

Group Members

Student ID	Student Name	GitHub ID
23904324	Adam Ahmadavi	aradavi63
24231774	Nikhil Chadha	nikhilchadha28
25039111	Nandhukrishna Gujjalapudi	nandhugnk
24771683	Cong Lu	cong53077-spec
24535749	Syeda Mehnaz Monsur	Mehnaz-M

GitHub Repository: https://github.com/aradavi63/CITS5501-G14

---

1. Task 1 – Prototype

1.1 What the Prototype Does

The prototype is implemented in Java using the built-in HttpClient library to send HTTP requests and the jsoup library to parse HTML. Its main purpose is to identify and attempt to download a small selection of policy documents from the supplied UWA policy website. Although our Phase 1 plan was to use Maven, we used a plain javac build with the jsoup jar included in the lib/ folder for this minimal prototype, to keep the setup simple.

The program follows four main steps.

1. Download the Index Page

The scraper starts by requesting the UWA policy index page:

https://uwa-policy.cits5501.arranstewart.io/policy/

The index page is requested once during a normal run. The program also accepts an alternative starting URL as a command-line argument, allowing it to be tested against a locally hosted page without making requests to the live website.

2. Extract Policy Document Links

After downloading the index page, we used jsoup to find all the links on it. Some of these were relative links, so we converted them into full URLs using the address of the index page.

We then only kept links that were on the same website and pointed to a document. To decide what counted as a document, we checked the file extension. The prototype accepts PDF, Word, RTF, Excel, PowerPoint and OpenDocument files.

Links to other websites were ignored, along with things like images and scripts. If the same document appeared more than once, it was only kept once. We also kept the scraper simple. It doesn't visit any other pages on the site or follow links inside the documents it downloads.

3. Download Selected Documents

The scraper attempts to download documents from the discovered links until five successful HTTP downloads have been recorded or the request limit is reached.

To reduce unnecessary requests to the server, the prototype:

- Makes no more than 20 HTTP requests in total.
- Waits 1 second between document downloads.
- Processes document links in the order they appear on the index page.
- Keeps downloaded content in memory temporarily rather than saving files permanently.

A download is reported as an HTTP success when the server returns status 200 and the response isn't empty. This indicates that the request completed successfully at the HTTP level but does not necessarily mean that the downloaded content is a valid or readable policy document.

4. Display Download Metadata

For each attempted document download, the prototype displays:

- Policy document URL.
- Final URL after any redirects.
- HTTP response status.
- Declared Content-Type.
- Downloaded size in bytes.
- First eight bytes of the response in hexadecimal format.
- HTTP download outcome.

Displaying the initial bytes provides a basic indication of the returned content. For example, PDF files commonly begin with %PDF-. However, the prototype only displays these bytes and does not use them to validate the document.

Observed Results

During the reported run against the supplied UWA policy index page, the prototype identified 216 distinct candidate document URLs and recorded five successful HTTP downloads using six requests in total.

All five downloaded documents in that run were PDFs.

A representative example from the program output is shown below.

Sample Output

```
Distinct candidate URLs: 216

Policy URL:
https://uwa-policy.cits5501.arranstewart.io/policy/-/media/project/uwa/uwa/
```

```
policy-library/policy/code-of-conduct/integrity/policy-framework/policy-
governance-framework.pdf

Final URL:
https://uwa-policy.cits5501.arranstewart.io/policy/-/media/project/uwa/uwa/
policy-library/policy/code-of-conduct/integrity/policy-framework/policy-
governance-framework.pdf

HTTP status: 200
Content-Type: application/pdf
Downloaded bytes: 301201
Initial bytes (hex): 255044462d312e36
HTTP transfer: SUCCESS

Summary: 5 distinct successful HTTP transfers; 6/20 requests
```

The output for the remaining four PDF downloads is omitted here for brevity.

Run Summary

Measurement	Reported Result
Distinct candidate URLs	216
Successful HTTP downloads	5
Document formats downloaded	PDF
Total HTTP requests	6
Maximum permitted requests	20

The results demonstrate that the prototype can identify candidate policy links and retrieve a small number of documents while remaining within the specified request limit. However, successful HTTP responses do not establish that the downloaded documents have been fully validated.

1.2 Assumptions

The following assumptions were made to keep the prototype within the scope of the Phase 2 requirements.

A001 – Documents Are Accessible from the Index Page

We assume that the required policy documents can be discovered directly from the supplied index page. Therefore, the prototype does not crawl additional webpages or follow links within documents. This is consistent with the scope defined in the project specification.

A002 – Document Links Contain Recognisable File Extensions

We assume that downloadable policy documents have supported extensions such as .pdf, .docx or .rtf. This allows the scraper to identify likely document links without inspecting every linked resource.

A003 – Policy Documents Are Hosted on the Same Website
We assume that the documents required for this prototype are hosted on the same website as the policy index. Links to external websites are excluded to keep the scraping process limited to the supplied target.

A004 – Document Links Are Present in the Retrieved HTML
We assume that policy document links are available in the HTML returned by the server and do not require JavaScript execution. This allows jsoup to extract links directly from the response.

A005 – The Index Page Uses UTF-8 Encoding
We assume that the index page uses UTF-8 encoding, allowing its HTML content to be processed consistently without additional character-encoding detection.

1.3 Limitations

1. Relying on File Extensions
The scraper decides whether a link is a document by looking at the end of the URL. This means some documents could be missed if their links don't end in a normal file extension. It can also go the other way. A link ending in .pdf might be picked up even if the server actually sends back an error page instead of a real PDF.

2. Not Much Checking of Downloads
If the server returns HTTP 200 with a non-empty response, the prototype counts the download as a success. It doesn't check whether the file is actually a proper document. The program does print the Content-Type and the first few bytes of the file, but it doesn't use them to check anything. So a broken PDF, or a response with the wrong label, could still show up as a success.

3. Only PDFs Were Tested in Practice
The scraper goes through the document links in the same order they appear on the index page. In the run we recorded, the first five successful downloads all happened to be PDFs. So even though the filter accepts Word, RTF and other formats, the results don't actually show that those formats download properly.

4. Basic Error Handling
If a document fails to download, the prototype doesn't try again, and it has no way of dealing with slow or offline servers. If the index page can't be loaded at all, the program just stops. Section 2 describes tests for how a production scraper should handle these cases.

5. No Recursive Crawling
The scraper only examines links available on the supplied index page. It does not follow links to additional webpages to discover documents. Therefore, documents accessible only through secondary pages would not be identified.

These limitations reflect the deliberately restricted scope of Task 1. A production scraper would require more comprehensive document validation, error handling and testing to operate reliably under unexpected website conditions.

2. Task 2 – Design of Test Suite

2.1 Overview

In this task, we design a set of tests for a full production level version of our scraper. Because a scraper depends on someone else's website, it has to cope with unexpected responses and failures, and it should never report misleading results when something goes wrong.

In Phase 1, our group identified website structure changes, connection failures, HTTP error responses, malformed documents and rate limiting as likely risks. The test cases below address these risks and the failure scenarios specified in the project brief.

We have designed ten test cases covering both recoverable and non-recoverable failures. These are designed tests for a fully functional production scraper, not executed test results, and they do not describe features of the Task 1 prototype.

2.2 Expected Behaviour

A good scraper doesn't need to fix every problem it runs into. What matters is that it notices when something goes wrong, reports it clearly, and never says a download worked when it didn't. Below is what we think a reasonable scraper should do in each of the five scenarios from the brief.

S1 – Website Structure Changes

The scraper should continue identifying valid document links when they remain accessible through standard HTML anchor elements. If no matching links are found, it should report that no policy documents were discovered rather than incorrectly reporting a successful scrape.

S2 – Truncated HTML

The scraper should process any complete document links that can be recovered, ignore incomplete links and terminate without crashing. Where truncation can be detected, it should report that the HTML response may be incomplete.

S3 – Timeouts

The scraper should retry temporary connection failures within a defined limit. If the timeout persists, it should stop attempting the request and report a timeout failure.

S4 – HTTP 4xx/5xx Errors

The scraper should report HTTP client errors, such as 403 and 404, without unnecessary retries. Temporary server errors, such as 503, may be retried within the configured limit.

S5 – Invalid Documents

The scraper should check the returned document type and content before accepting a download. Responses containing HTML instead of a PDF, or corrupted PDF files, should be rejected and reported as validation failures.

To make these expectations testable, we assume the production scraper behaves as follows.

TA001 – Timeout and Retry Handling
The scraper uses configurable connection and response timeouts. For this test suite, we assume a maximum of three attempts per request, consisting of one initial attempt and up to two retries for temporary failures.

TA002 – Document Link Discovery
The scraper identifies policy documents using supported links and file formats. It should continue to recognise valid document links when their location within the HTML page changes, provided they remain accessible through standard anchor elements.

TA003 – HTTP Error Handling
HTTP client errors, such as 403 and 404, are reported without unnecessary retries. Temporary server errors, such as 503, may be retried within the configured limit.

TA004 – Document Validation
Downloaded documents are checked before being treated as successfully retrieved. A file extension or HTTP 200 response alone is not sufficient evidence that the returned content is a valid document.

2.3 Test Cases

The following ten test cases use the Arrange–Act–Assert approach. Each test is labelled with the scenario (S1–S5) from Section 2.2 that it relates to.

T001 – Website Structure Changes

Field	Description
Test ID	T001
Scenario	S1 – Website structure changes
Arrange (Test Setup)	Create an HTML fixture where valid policy document links are moved to a different HTML container but remain accessible through standard anchor elements.
Act	Run the scraper against the modified page.
Assert (Expected Result)	All expected document URLs should be discovered despite the structural change. No unrelated links should be included.

T002 – No Policy Document Links

Field	Description
Test ID	T002
Scenario	S1 – Website structure changes
Arrange (Test Setup)	Provide an HTML page containing navigation links, images and other webpage resources but no downloadable policy documents.
Act	Run the document discovery process.
Assert (Expected Result)	The scraper should return an empty collection of document candidates and report that no matching policy links were found. It should not download unrelated resources.

T003 – Partially Truncated HTML

Field	Description
Test ID	T003
Scenario	S2 – Truncated HTML
Arrange (Test Setup)	Create an HTML fixture containing one complete, valid PDF link followed by a truncated anchor element with an incomplete href value that does not form a valid document URL.
Act	Run the scraper against the incomplete HTML page.
Assert (Expected Result)	The scraper should identify the complete PDF link, ignore the incomplete anchor and finish without an unhandled exception or invalid document URL.

T004 – Truncated HTML Without Valid Links

Field	Description
Test ID	T004
Scenario	S2 – Truncated HTML
Arrange (Test Setup)	Provide an HTML response that ends before any complete policy document link appears.
Act	Run the document discovery process.
Assert (Expected Result)	The scraper should terminate safely and report that no usable policy links were discovered. No successful document download should be reported.

T005 – Temporary Timeout

Field	Description
Test ID	T005
Scenario	S3 – Timeout (short)
Arrange (Test Setup)	Configure a controlled HTTP server to delay the first document request beyond the configured timeout. Return HTTP 200 with a valid PDF on the second request.
Act	Attempt to download the document.
Assert (Expected Result)	The scraper should detect the timeout, retry and successfully retrieve the valid PDF on the second attempt. Exactly two attempts should be recorded.

T006 – Persistent Timeout

Field	Description
Test ID	T006
Scenario	S3 – Timeout (long)
Arrange (Test Setup)	Configure a controlled HTTP server to delay every document request beyond the configured timeout.
Act	Attempt to download the document.
Assert (Expected Result)	The scraper should stop after three total attempts, report a timeout failure and avoid reporting a successful download.

T007 – HTTP 4xx Errors

Field	Description
Test ID	T007
Scenario	S4 – HTTP 4xx errors
Arrange (Test Setup)	Configure two document URLs to return HTTP 403 (Forbidden) and HTTP 404 (Not Found), respectively.
Act	Request both document URLs.
Assert (Expected Result)	The scraper should record the corresponding status codes, reject both responses and avoid unnecessary retries. Each URL should be requested only once.

T008 – HTTP 5xx Errors

Field	Description
Test ID	T008
Scenario	S4 – HTTP 5xx errors
Arrange (Test Setup)	Configure a controlled HTTP server to return HTTP 503 for the first two requests and HTTP 200 with a valid PDF on the third attempt.
Act	Attempt to download the document.
Assert (Expected Result)	The scraper should retry within the configured limit and successfully retrieve the PDF on the third attempt. Exactly three attempts should be recorded.

T009 – HTML Returned Instead of PDF

Field	Description
Test ID	T009
Scenario	S5 – HTML instead of PDF
Arrange (Test Setup)	Configure a .pdf URL to return HTTP 200 with an HTML error page and Content-Type: text/html.
Act	Attempt to download the file.
Assert (Expected Result)	The scraper should reject the response because its Content-Type is text/html and its initial bytes do not match the expected %PDF-signature. It should report a document-type validation failure.

T010 – Corrupted PDF

Field	Description
Test ID	T010
Scenario	S5 – Corrupted PDF
Arrange (Test Setup)	Provide a PDF response beginning with a valid %PDF- header but missing essential PDF structures due to truncation.
Act	Download and validate the document.
Assert (Expected Result)	The scraper should identify the corrupted PDF, report a validation failure and avoid treating the file as a successful document download.

2.4 Testing Approach

We would use JUnit 5 to implement the proposed test cases, consistent with the testing framework selected in Phase 1. Maven would be used to manage the project and execute the tests.

Most tests would run against local HTML fixtures and a controlled HTTP server instead of the live website. This would allow failures such as timeouts, HTTP errors and invalid document responses to be reproduced reliably. Each fixture would have a predefined expected outcome, allowing the tests to be evaluated independently of the live website. Fixtures and test doubles are discussed in more detail in Task 3.

2.5 Test Evaluation

A test would be considered successful when the scraper behaves according to the expected result defined in the corresponding test case.

For example, rejecting an invalid PDF, reporting an HTTP 404 response correctly or stopping after three unsuccessful timeout attempts would be considered correct behaviour.

A test would fail if the scraper:

- Crashes because of malformed HTML or an unexpected response.
- Reports a successful download when the returned content is invalid.
- Continues retrying beyond the configured limit.
- Downloads unrelated webpage resources as policy documents.
- Ignores HTTP errors or provides misleading results.
- Fails to identify valid document links that remain accessible through supported HTML elements.

Some website changes may prevent document discovery entirely. In such cases, a controlled failure with a meaningful error message is preferable to incorrectly reporting that the operation succeeded.

2.6 Summary

The ten proposed test cases cover all five failure scenarios identified in the project specification. They address website structure changes, truncated HTML, temporary and prolonged timeouts, HTTP 4xx and 5xx responses, and invalid or corrupted documents.

The test suite focuses on common failure conditions that could affect the reliability of a production scraper. Additional cases involving duplicate links, redirects, unusually large files and other adversarial inputs can be considered separately in Task 4.

Using controlled fixtures and test doubles would allow the team to reproduce these scenarios reliably and evaluate whether a production scraper handles failures as expected.

---

3. Task 3 – Fixtures and Test Doubles

1. What kind of fixtures would you use as a "source of truth" for whether your scraper is working correctly?

Test fixtures are prepared files we use to test our web scraper in a controlled environment. They can work as a source of truth when fixture contains known information or have expected results which we can compare with scraper's actual output.

For our scraper we can prepare HTML file containing known numbers of policy documents links, or a PDF file to test how scraper handles both valid and corrupted files. As we know what the fixtures contains, we can easily determine our output is correct or not. The known expected result acts as our test oracle.

Even if there are any changes on real website our fixtures remain the same so will work as a source of truth and we can use them repeatedly.

2. Does the target site seem to limit high levels of requests? Describe your observations.

Rate limiting is when a website restricts the number of requests that can be sent to a website by scraper in a particular time frame.

To test whether the target university policy website has rate limits or not we used curl to send one HEAD request. The website returned HTTP/1.1 200 OK, along with X-Ratelimit-Limit: 3, X-Ratelimit-Remaining: 2, and Retry-After: 10.

The headers show that the site allows three requests in a short period of time, and we hit this limit ourselves. When our prototype waited only half a second between downloads, the site blocked the fifth document with an HTTP 429 "Too Many Requests" response. This can be seen in RUN-OUTPUT-0.5s.txt. After we changed the wait to one second, all six requests went through without being blocked. We didn't test exactly how long the site makes you wait before the limit resets.

To test our scraper safely we can use a fake local server (test double) to simulate HTTP 429 responses after certain requests. This will allow us to test rate limit and waits before retrying correctly without putting extra load on real website.

3. Can test doubles be used to simulate timeouts, interrupted downloads, connection failures, or very slow responses? What tools or frameworks are available in your chosen language for doing this?

Test doubles can be used to simulate timeouts, interrupted downloads, connection failures, or very slow responses during testing a scraper downloading policy documents.

For example, a fake local server can deliberately take longer to respond than the scraper's timeout or can interrupt the internet connection during the PDF download or tries to connect to wrong port.

Tools we can use in Java are Java HTTP Server to create a fake local HTTP server, Mockito to create mock objects to simulate behaviours and WireMock to simulate HTTP responses, delays, and failures.

Using test doubles over real servers are more useful as in that case we can repeat same tests, we can control when failures happen, we do not need to cause problems on real website, and some failures are difficult to occur naturally.

Demonstration

For the practical demonstration we created a Java test double named InterruptedDownloadTestDouble.java and used HttpServer to create a fake local server. The server runs at http://localhost:8080/FakePdf.pdf.

The fake server simulates an incomplete PDF download. In that test it tells the client to expect 1054 bytes. However, it deliberately stops after sending 54 bytes.

To get this we used curl to request fake PDF and configured a 5 second timeout. During test output confirmed that only 54 of 1054 bytes were received.

We used test double because in it we can control when the failure happens, can repeat the same test and avoid putting unnecessary load on the real university website.

The same scenario is difficult or undesirable to reproduce against live site because interrupted download is difficult to reproduce reliably on a real server.

---

4. Task 4 – Adversarial Inputs

These are tests that challenge assumptions about how document links, redirects, downloads and server responses should behave, as if we were testing another group's scraper.

The ideas are based on the risks identified in Phase 1, the assumptions documented in Task 1, and the implementation of our Java prototype. For example, the prototype identifies documents using file extensions, compares final URLs to determine distinctness, assumes UTF-8 encoding and temporarily stores complete response bodies in memory.

The following tables present the adversarial test designs. Each test uses a controlled local server with predefined content and expected outcomes.

ADV001 – Relative and Unusual URLs

Field	Description
Test ID	ADV001
Assumption being challenged	Document links are simple absolute URLs with lowercase extensions and no query parameters or encoded characters.
Input artifact and testing method	Serve an index at /policy/index.html containing docs/POLICY.PDF?version=2, ../docs/leave%20policy.docx and /docs/c.rtf. Provide valid documents at the correctly resolved locations. Run the scraper against this index and record its requests.
Failure and how it would be detected	Failure: The scraper misses a document or requests the wrong URL. Correct: It retrieves all three documents using the expected paths and preserves query parameters. Detection: Compare the server's request log and scraper output with the predefined URLs.

ADV002 – Duplicate Documents After Redirects

Field	Description
Test ID	ADV002
Assumption being challenged	Different final URL strings always represent distinct documents.
Input artifact and testing method	Create five document links that redirect to /same.pdf#0 through /same.pdf#4. All destinations return the same resource. Run the scraper and compare its reported distinct-document count with the fixture's expected count.
Failure and how it would be detected	Failure: The scraper counts the same document more than once because the URL fragments differ. Correct: It reports one distinct document. Detection: Compare its reported count with the server log, which shows that the redirects lead to the same resource.

ADV003 – Continuously Changing Redirects

Field	Description
Test ID	ADV003
Assumption being challenged	Every endless redirect sequence eventually repeats an identical URL and can therefore be detected using a visited-URL set.
Input artifact and testing method	Configure /a.pdf to redirect to /a.pdf?step=1, then ?step=2, continuing without repeating a URL. Set a total budget of 20 HTTP requests, including the index request. Run the scraper and count requests.
Failure and how it would be detected	Failure: The scraper exceeds 20 requests, keeps running or incorrectly reports a successful download. Correct: It stops within the request limit and reports that no document was downloaded. Detection: Count requests in the server log and check the scraper's final output and termination.

ADV004 – Redirect Outside the Permitted Origin

Field	Description
Test ID	ADV004
Assumption being challenged	A document link on the permitted website remains within that website after redirection.
Input artifact and testing method	Start two local servers on different ports. Server A hosts the index and a document endpoint that redirects to server B. Configure A as the only permitted origin, run the scraper and inspect both request logs.
Failure and how it would be detected	Failure: The scraper sends a request to server B. Correct: It refuses the redirect and reports that the download was unsuccessful. Detection: Check that server B's request log is empty and that the scraper explains why it stopped.

ADV005 – Large Download Without a Size Header

Field	Description
Test ID	ADV005
Assumption being challenged	Documents are small enough to load into memory, or their size can always be checked using Content-Length.
Input artifact and testing method	Stream a known valid 20 MiB PDF without a Content-Length header. Configure the production scraper with a 5 MiB download limit. Run the download and use a byte counter in the receiving stream to measure consumption.
Failure and how it would be detected	Failure: The scraper downloads the entire oversized file, accepts it or crashes. Correct: It stops near the configured 5 MiB limit and reports a size-limit failure. Detection: Measure the bytes received and check that the document is not counted as successful.

ADV006 – Response Body Stalls After Headers Arrive

Field	Description
Test ID	ADV006
Assumption being challenged	A request timeout necessarily limits the time required to receive the complete document.
Input artifact and testing method	Immediately return HTTP 200 headers declaring the correct body length, wait 23 seconds, then send the body. Configure a 20-second whole-transfer deadline with a two-second scheduling tolerance. Measure the document request's duration.
Failure and how it would be detected	Failure: The scraper continues beyond the deadline and tolerance or accepts the late document. Correct: It stops within 22 seconds and reports a timeout. Detection: Measure the document request's duration and check the reported outcome.

ADV007 – Rate Limiting With a Waiting Instruction

Field	Description
Test ID	ADV007
Assumption being challenged	An unsuccessful request can be retried immediately, or a fixed one-second pause always satisfies the server's restrictions.
Input artifact and testing method	Return HTTP 429 with Retry-After: 2 for the first document request. Permit a successful response after two seconds. Before running the test, record whether the scraper is configured to stop or retry, with a maximum of three attempts. Capture request timestamps and outcomes.
Failure and how it would be detected	Failure: The scraper retries too early, exceeds three attempts or counts the 429 response as a successful download. Correct: It clearly reports the rate limit and stops, or waits at least two seconds before retrying within the attempt limit. Detection: Check request timestamps, attempt counts and scraper output against the selected policy.

ADV008 – Non-UTF-8 Index Encoding

Field	Description
Test ID	ADV008
Assumption being challenged	Every index page uses UTF-8, so decoding its bytes as UTF-8 preserves document links.
Input artifact and testing method	Serve a Windows-1252 index with Content-Type: text/html; charset=windows-1252. Include a literal café.pdf link and provide the intended document at /caf%C3%A9.pdf. Run the scraper against the index at the origin root.
Failure and how it would be detected	Failure: The scraper corrupts the URL, requests the wrong path or silently misses the document. Correct: It retrieves the intended document or clearly reports that the encoding is unsupported. Detection: Compare requested paths with the expected URL and check the scraper's diagnostics.

Each test would start with a fresh server state and scraper process. We would provide the local index through the scraper's command-line argument and capture its output alongside server request logs. Assertions would use the predefined document set, request counts, timestamps and measured outcomes rather than relying only on whether the program produced output or exited normally.

The size limit, whole-transfer deadline and retry settings above are proposed configurations for a production scraper. They do not describe features already implemented by the minimal prototype. Some tests may confirm that existing safeguards work correctly; an adversarial test does not need to expose a defect to provide useful evidence.